# Advanced Strings

## Suffix array + LCP, suffix automaton, and the problems they kill

# Learning goals

- Build a suffix array in O(n log n) and explain *why* doubling works
- Compute the LCP array with Kasai in O(n) and prove the amortization
- Turn "substring questions" into **SA ranges** and **LCP range minima**
- Build a suffix automaton online and reason about `endpos` classes
- Use the suffix-link tree for occurrence counts, LCS, k-th substring
- Pick SA vs SAM vs hashing under contest pressure

Prerequisites: Year weeks 28–29 (hashing, KMP, Z, trie, SA intro), Regional 09

# Motivating problem

> Given `s` ($|s| \le 10^5$). For every **distinct** substring `t`, let `occ(t)` be its number of occurrences. Output $\sum \text{occ}(t)\cdot(\text{occ}(t)+1)/2$.

(Codeforces 123D *String*)

- $O(n^2)$ distinct substrings — cannot enumerate
- Hashing gives membership, not structure
- Need a structure where *all substrings with equal occurrence sets* are grouped

Both the SA+LCP and the SAM solve this in $O(n \log n)$ / $O(n)$.

# Suffix array: definition

`sa[i]` = start of the i-th smallest suffix. `rank[sa[i]] = i`.

```
s = banana          i  sa  suffix     lcp(i,i+1)
                    0   5  a          1
                    1   3  ana        3
                    2   1  anana      0
                    3   0  banana     0
                    4   4  na         2
                    5   2  nana       -
```

Key fact: suffixes sharing a prefix `p` form a **contiguous block** of `sa`.

# Doubling: sort cyclic shifts of `s + '\0'`

- Let `c_k[i]` = class of the length-$2^k$ substring starting at `i` (cyclic)
- Length-$2^{k+1}$ substring at `i` = pair `(c_k[i], c_k[i + 2^k])`
- Sort pairs → new classes. After ⌈log n⌉ rounds, classes = suffix order
- The sentinel `'\0'` makes cyclic order = suffix order

Radix trick: sorting by the second key is free —
`p[i] - 2^k` already lists shifts in order of their second half.
Then one **stable counting sort** by first key ⇒ O(n) per round.

Total: **O(n log n)**; stop early when all classes are distinct.

# Suffix array code

```
vector<int> suffixArray(const string& str) {
    string s = str + char(0); int n = s.size(), A = 256;
    vector<int> p(n), c(n), pn(n), cn(n), cnt(max(A, n), 0);
    for (char ch : s) cnt[(unsigned char)ch]++;
    for (int i = 1; i < A; i++) cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; i--) p[--cnt[(unsigned char)s[i]]] = i;
    int classes = 1; c[p[0]] = 0;
    for (int i = 1; i < n; i++) { if (s[p[i]] != s[p[i-1]]) classes++; c[p[i]] = classes - 1; }
    for (int h = 1; h < n && classes < n; h <<= 1) {
        for (int i = 0; i < n; i++) pn[i] = (p[i] - h + n) % n;
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0; classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int,int> a = {c[p[i]], c[(p[i] + h) % n]}, b = {c[p[i-1]], c[(p[i-1] + h) % n]};
            if (a != b) classes++;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
    }
    p.erase(p.begin()); return p;   // drop the sentinel
}
```

# Kasai's LCP algorithm

`lcp[i] = LCP(sa[i], sa[i+1])`. Process suffixes in **text order** `i = 0..n-1`.

**Lemma.** If suffix `i` has LCP `k > 0` with its successor in `sa`, then suffix `i+1` has LCP ≥ `k − 1` with *its* successor.

*Proof.* Let `j` follow `i` in sa. Then `j+1` is a suffix with `s[j+1..] > s[i+1..]` sharing `k−1` chars. Everything sorted between them shares ≥ `k−1` chars too. ∎

So `k` decreases by ≤ 1 per step and never exceeds n ⇒ **O(n)** total.

# Kasai code

```
vector<int> lcpArray(const string& s, const vector<int>& sa) {
    int n = s.size();
    vector<int> rk(n), lcp(max(n - 1, 0));
    for (int i = 0; i < n; i++) rk[sa[i]] = i;
    for (int i = 0, k = 0; i < n; i++) {
        if (k > 0) k--;
        if (rk[i] == n - 1) { k = 0; continue; }
        int j = sa[rk[i] + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
        lcp[rk[i]] = k;
    }
    return lcp;
}
```

Pair it with a sparse table: `LCP(any two suffixes) = min lcp[rk_a .. rk_b − 1]` in O(1).

# SA + LCP: the standard toolkit

| Question | Answer |
|---|---|
| Distinct substrings | $n(n+1)/2 - \Sigma$ lcp |
| Longest repeated substring | max lcp |
| Occurrences of pattern p | binary search block in sa, $O(m \log n)$ |
| LCP of suffixes a, b | RMQ on lcp between ranks |
| Longest substring occurring ≥ k times | max over windows of $k-1$ lcp's of their min |
| LCS of s, t | SA of `s # t`, adjacent pair from different sides |
| k-th distinct substring | walk sa, suffix i contributes $n - sa[i] - lcp[i-1]$ new ones |

# The "LCP interval" view

Group suffixes by LCP ≥ L: maximal sa-ranges where all inner lcp ≥ L.

- Each such range = **one substring of length L** (and its occurrence count = range size)
- Ranges for all L form a tree = the **suffix tree** (internal nodes)
- Enumerate with a monotonic stack over lcp (Year week 12!) in O(n)

CF 123D: for each internal node with depth range (parent depth, depth] and size m,
add (depth − parentDepth) · m(m+1)/2; leaves contribute their own lengths.

# Pattern search with SA

```cpp
// returns [lo, hi) range of sa whose suffixes start with p
pair<int,int> findRange(const string& s, const vector<int>& sa, const string& p) {
    auto cmp = [&](int start, const string& pat) {   // suffix < pat ?
        return s.compare(start, pat.size(), pat) < 0;
    };
    int lo = lower_bound(sa.begin(), sa.end(), p, cmp) - sa.begin();
    int hi = upper_bound(sa.begin(), sa.end(), p, [&](const string& pat, int start) {
        return s.compare(start, pat.size(), pat) > 0;
    }) - sa.begin();
    return {lo, hi};
}
```

O(m log n) per query. Answering many patterns: count = `hi - lo`, positions = `sa[lo..hi)`.

# Suffix automaton: `endpos` classes

`endpos(t)` = set of end positions of occurrences of `t` in `s`.

- Group substrings with equal `endpos` → a **state**
- In a state, strings are suffixes of each other, lengths form an interval `(len(link), len]`
- **Suffix link** of v → state of the longest suffix of v's strings in a *different* class
- Links form a tree rooted at the empty string; endpos sets are **nested** along it

Facts: ≤ 2n − 1 states, ≤ 3n − 4 transitions (n ≥ 3). Construction is **online**, amortized O(n·σ) with arrays or O(n log σ) with maps.

# SAM picture: `s = abb`

```
states (len, link)            transitions
0: ""       (0, -)            0 -a-> 1, 0 -b-> 4
1: a        (1, 0)            1 -b-> 2
2: ab       (2, 4)            2 -b-> 3
3: abb,bb   (3, 4)            4 -b-> 3
4: b        (1, 0)   <- clone

endpos(b) = {2,3}, endpos(ab) = {2}, endpos(bb)=endpos(abb) = {3}
```

Clone 4 was created when adding the second `b`: `b` started occurring at two ends, `ab` only at one.

# SAM extend

```
void extend(char ch) {
    int c = ch - 'a', cur = newState(st[last].len + 1), p = last;
    while (p != -1 && st[p].next[c] == -1) { st[p].next[c] = cur; p = st[p].link; }
    if (p == -1) st[cur].link = 0;
    else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) st[cur].link = q;
        else {
            int clone = newState(st[p].len + 1);
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c] == q) { st[p].next[c] = clone; p = st[p].link; }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}
```

Three cases: no `c`-edge anywhere / **solid** edge (len+1) / **split** with clone.

# Why is extend amortized O(1)?

- Loop 1 adds new transitions — total transitions are O(n)
- Loop 2 (redirect to clone): track `len(link(last))`; potential argument shows the
  total number of redirections is O(n) (the depth of `link(last)` in the suffix-link tree drops)
- Number of states ≤ 2n − 1 because each step makes ≤ 2 states and the first two steps make 1

Memory: `array<int,26>` × 2n states = 52n ints. For n = $10^6$ that is ~200 MB — use `map`/sorted edge lists or a smaller alphabet when n is large.

# SAM toolkit

| Question | How |
|---|---|
| Is p a substring? | walk transitions from root |
| Distinct substrings | Σ (len[v] − len[link[v]]) |
| occ(t) = \|endpos\| | weight 1 on prefix states, sum up the link tree |
| First occurrence | `firstpos`, set on create, copied on clone |
| LCS(s, t) | walk t on SAM(s), fall back by links |
| k-th substring | DP: #paths from each state, then greedy walk |
| Shortest absent string | DP over transitions |

# Counting occurrences

```
// occ[v] = |endpos(v)|.  Prefix states get 1, clones get 0.
int v = 0;
for (char ch : s) { v = st[v].next[ch - 'a']; occ[v] = 1; }
// process states by decreasing len (counting sort), push to link
for (int i = m - 1; i > 0; i--) occ[st[order[i]].link] += occ[order[i]];
```

CF 123D on SAM: `answer = Σ_v (len[v] − len[link[v]]) · occ[v](occ[v]+1)/2`.

CF 802I *Fake News (hard)*: same, with `occ²`.

# LCS of two strings (SPOJ LCS)

Walk `t` over SAM(s), maintaining `(v, l)` = longest suffix of `t[..i]` that is a substring of `s`.

```
int v = 0, l = 0, best = 0;
for (char ch : t) {
    int c = ch - 'a';
    while (v != 0 && st[v].next[c] == -1) { v = st[v].link; l = st[v].len; }
    if (st[v].next[c] != -1) { v = st[v].next[c]; l++; }
    best = max(best, l);
}
```

Many strings (SPOJ LCS2): for each state keep the min over strings of the max match,
propagating the match up the link tree.

# k-th lexicographic substring

- `paths[v]` = number of non-empty paths from v (distinct) — DAG DP in reverse len order
- Greedy from root: for letters a..z with u = next[c]: if k ≤ 1 + paths[u], take c (k −= 1; stop if k = 0, else continue at u); otherwise k −= 1 + paths[u]
- Variant **with repetitions** (CF 128B, CSES 2109): weight each step by `occ[next]`

The SA version: suffix `sa[i]` contributes substrings of lengths `lcp[i−1]+1 .. n−sa[i]`
— scan sa until the running total reaches k (distinct version, CSES 2108).

## Suffix-link tree = suffix tree of reverse

- Suffix links of SAM(s) form the suffix tree of `reverse(s)`
- So DSU-on-tree / small-to-large (Lecture 4) on the link tree = processing endpos sets
- **Merging endpos sets** with segment-tree merge answers "does t occur inside s[l..r]"
  (CF 1037H *Security*, CF 666E *Forensic Examination*)

Persistent/mergeable segment trees: Lecture 2.

# SA vs SAM vs hashing

| | SA + LCP | SAM | Hashing |
|---|---|---|---|
| Build | O(n log n) | O(n·σ) | O(n) |
| Memory | small | large (σ arrays) | small |
| Lexicographic order | natural | via DFS | painful |
| Online append | no | **yes** | yes |
| Many-string problems | concatenate | generalized SAM | easy |
| Bug risk | low | medium | collision |

Rule of thumb: **order / ranges ⇒ SA**; **counting / online ⇒ SAM**.

# Complexity summary

| Operation | Cost |
|---|---|
| SA (doubling + radix) | O(n log n) time, O(n) memory |
| SA-IS / DC3 | O(n) (rarely needed in contests) |
| Kasai | O(n) |
| LCP of any two suffixes | O(1) after O(n log n) sparse table |
| SAM build | O(n) states/edges, O(nσ) memory with arrays |
| Occurrence counts | O(n) with counting sort by len |

# Common pitfalls

- Forgetting the sentinel → cyclic shifts ≠ suffixes (`aaa` breaks)
- `char` is signed: index `cnt[(unsigned char)c]`
- Kasai: off-by-one between `lcp[i]` (pair i, i+1) vs `lcp[i]` (pair i−1, i)
- SA of concatenation: separator must be **unique** and not in the alphabet
- SAM: clone must copy `next` **and** `link`; occ of clones starts at 0
- SAM memory blow-up; `reserve(2n)` to avoid reallocation invalidating references
- Answer overflow: number of substrings ~ $n^2/2$ needs `long long`

# Practice problems

- CSES 2105 *Distinct Substrings*, 2106 *Repeating Substring*
- CSES 2108 / 2109 *Substring Order I / II*, 2110 *Substring Distribution*
- SPOJ LCS / LCS2 *Longest Common Substring (II)*
- Codeforces 123D *String*, 128B *String*, 802I *Fake News (hard)*
- Codeforces 316G3 *Good Substrings*, 427D *Match & Catch*
- Codeforces 873F *Forbidden Indices*, 235C *Cyclical Quest*
- Stretch: Codeforces 1037H *Security*, 666E *Forensic Examination*

# Summary

- **SA**: sorted suffixes; substrings with a common prefix = contiguous blocks
- **LCP + RMQ**: compares any two suffixes in O(1); intervals = suffix-tree nodes
- **SAM**: minimal DFA of all substrings; states = endpos classes; links = tree
- Most national-level string problems reduce to *aggregate over a tree of substring classes*
- Next lecture: matrices, FFT/NTT, Gaussian elimination